



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Отчёт о лабораторной работе № 1

по курсу «Разработка параллельных и распределенных программ»

Студент: Ивенкова О. В.

Группа: ИУ9-52Б

Преподаватель: Царёв А. С.

Москва, 2025

Содержание

1	Постановка задачи	3
2	Практическая реализация	4
3	Сравнение вычислений	8
4	Характеристики устройства	8

1 Постановка задачи

Лабораторная работа №1: распараллеливание алгоритма вычисления произведения двух матриц.

Две квадратные матрицы A и B размерности n сначала перемножить стандартным алгоритмом:

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj} \quad (1)$$

для получения матрицы C той же размерности. Замерить время вычисления, сравнить с временем при вычислении элементов матрицы C не по строкам, а по столбцам. Размер матриц подобрать таким образом, чтобы время выполнения на вашей машине было не слишком непоказательно малым (меньше нескольких минут), но и не чересчур большим (несколько часов). Использовать библиотечные функции для вычисления произведений матриц нельзя.

Затем конечную матрицу C условно разделить на примерно равные прямоугольные подматрицы и распараллелить программу таким образом, чтобы каждый поток занимался вычислением своей подматрицы. Матрицы A и B для этого разделить на примерно равные группы строк и столбцов соответственно. Сделать для разного количества потоков (разных разбиений), также замерить время вычисления, сравнить с вычислениями стандартным алгоритмом. Также по окончании вычислений сравнивать получившуюся матрицу с той, что была вычислена стандартным алгоритмом, для проверки правильности вычислений (проверка во время выполнения задачи не входит).

Уделить внимание проблеме синхронизации потоков: основной поток, который создает потоки, занимающиеся вычислением подматриц, не должен закончиться до того, как они закончат свою работу, поэтому организовать ожидание и сигнализацию завершения рабочих потоков.

При наличии в языке программирования и архитектуре возможностей управления приоритетами процессов и потоков оценить также их влияние на время работы программы.

После очной сдачи программы подготовить отчёт по лабораторной работе. Титульный лист по стандартному образцу, остальное в свободной форме, но обязательно привести:

- листинг текста программы
- характеристики устройства, на котором производились вычисления
- времена работы программы как на стандартном, непараллельном алгоритме, так и с использованием многопоточности (в виде числовых значений, а также в виде графика, где по оси абсцисс число потоков, а по оси ординат – время вычислений)
- при наличии возможности управления приоритетами процессов и потоков также сравнение времен вычислений при использовании высокоприоритетных или низкоприоритетных процессов и потоков с нормальным приоритетом.

2 Практическая реализация

Листинг 1: Код программы

```

1 package Lab1.java;
2
3 import java.util.Random;
4 import java.util.concurrent.CountDownLatch;
5
6 public class MatrixMultiply {
7     static double [][] randomMatrix(int n, long seed) {
8         Random rnd = new Random(seed);
9         double [][] M = new double[n][n];
10        for (int i = 0; i < n; ++i) {
11            for (int j = 0; j < n; ++j) {
12                M[i][j] = rnd.nextDouble();
13            }
14        }
15        return M;
16    }
17
18    static double [][] multiplyRowMajor(double [][] A, double [][] B) {
19        int n = A.length;
20        double [][] C = new double[n][n];
21        for (int i = 0; i < n; ++i) {
22            for (int k = 0; k < n; ++k) {
23                double aik = A[i][k];
24                for (int j = 0; j < n; ++j) {

```

```

25         C[i][j] += aik * B[k][j];
26     }
27 }
28 }
29 return C;
30 }
31
32 static double [][] multiplyColMajor(double [][] A, double [][] B) {
33     int n = A.length;
34     double [][] C = new double[n][n];
35     for (int j = 0; j < n; ++j) {
36         for (int k = 0; k < n; ++k) {
37             double bkj = B[k][j];
38             for (int i = 0; i < n; ++i) {
39                 C[i][j] += A[i][k] * bkj;
40             }
41         }
42     }
43     return C;
44 }
45
46 static double [][] multiplyParallelBlocks(double [][] A, double [][] B,
47 int numThreads) throws InterruptedException {
48     int n = A.length;
49     double [][] C = new double[n][n];
50     CountDownLatch latch = new CountDownLatch(numThreads);
51
52     int rowsPerThread = n / numThreads;
53
54     for (int t = 0; t < numThreads; ++t) {
55         int startRow = t * rowsPerThread;
56         int endRow = (t == numThreads - 1) ? n : (startRow +
57 rowsPerThread);
58
59         Thread worker = new Thread(() -> {
60             for (int i = startRow; i < endRow; ++i) {
61                 for (int k = 0; k < n; ++k) {
62                     double aik = A[i][k];
63                     double [] Bk = B[k];
64                     double [] Ci = C[i];
65                     for (int j = 0; j < n; ++j) {
66                         Ci[j] += aik * Bk[j];
67                     }
68                 }
69             }
70             latch.countDown();

```

```

69         });
70         worker.start();
71     }
72
73     latch.await();
74     return C;
75 }
76
77 static boolean equalMatrices(double[][] X, double[][] Y, double eps)
78 {
79     int n = X.length;
80     for (int i = 0; i < n; ++i) {
81         for (int j = 0; j < n; ++j) {
82             if (Math.abs(X[i][j] - Y[i][j]) > eps) {
83                 System.err.printf("Mismatch at [%d,%d]: %g vs %g%n",
84                     i, j, X[i][j], Y[i][j]);
85                 return false;
86             }
87         }
88     }
89     return true;
90 }
91
92 static long timeMs(Runnable r) {
93     long t0 = System.nanoTime();
94     r.run();
95     long t1 = System.nanoTime();
96     return (t1 - t0) / 1_000_000;
97 }
98
99 public static void main(String[] args) throws InterruptedException {
100     int n = 2000;
101
102     double[][] A = randomMatrix(n, 12345L);
103     double[][] B = randomMatrix(n, 54321L);
104
105     long tRow = timeMs(() -> {
106         double[][] C1 = multiplyRowMajor(A, B);
107     });
108     System.out.printf("Single-thread row-major multiply: %d ms%n",
109         tRow);
110
111     long tCol = timeMs(() -> {
112         double[][] C2 = multiplyColMajor(A, B);
113     });

```

```

111         System.out.printf("Single-thread col-major multiply: %d ms%n",
112                               tCol);
113
114         double [][] C_ref = multiplyRowMajor(A, B);
115
116         int [] threadTests = {2, 4, 8};
117         for (int t : threadTests) {
118             final int threads = t;
119             long tPar = timeMs(() -> {
120                 try {
121                     double [][] Cpar = multiplyParallelBlocks(A, B,
122                               threads);
123                     if (!equalMatrices(Cpar, C_ref, 1e-9)) {
124                         System.err.println("Error: parallel result not
125 equal with etalon!");
126                     }
127                 } catch (InterruptedException e) {
128                     throw new RuntimeException(e);
129                 }
130             });
131             System.out.printf("Parallel multiply (%d threads): %d ms%n",
132                               threads, tPar);
133         }
134     }
135 }

```

3 Сравнение вычислений

Листинг 2: Вывод после запуска кода

```
1 Single-thread row-major multiply: 9795 ms
2 Single-thread col-major multiply: 210383 ms
3 Parallel multiply (2 threads): 5266 ms
4 Parallel multiply (4 threads): 3705 ms
5 Parallel multiply (8 threads): 2308 ms
6 Done
```

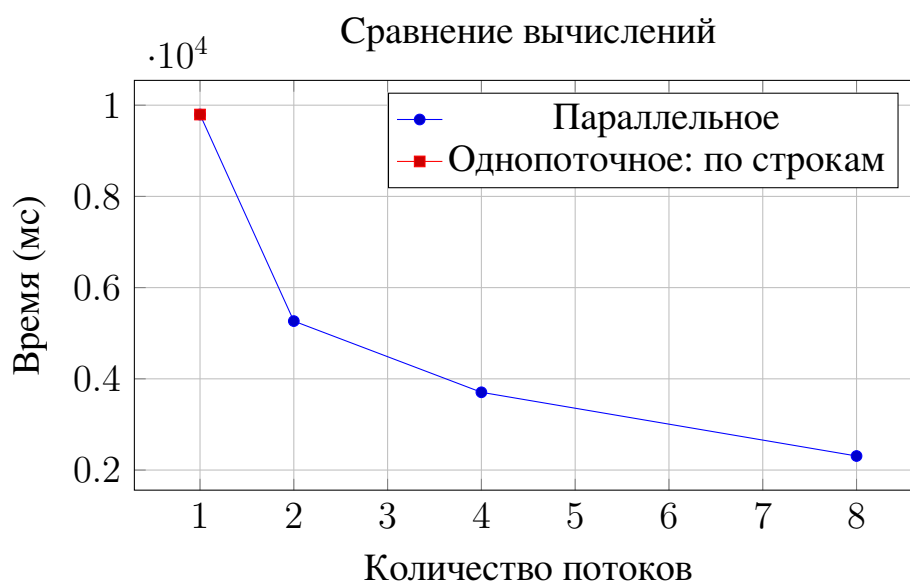


Рис. 1 — Сравнение времени выполнения различных реализаций

4 Характеристики устройства

Имя устройства Acer Extensa 215-54

Оперативная память 16,0 ГБ

Процессор устройства Intel Core i5-1135G7 (2.40 ГГц, 4 ядра, 15 Вт)